СПРАВОЧНИК · АРТУР ХАНТ

Основы Python с нуля — шпаргалка с примерами

Как установить и запустить Python, 16 уроков «правило → пример → результат», готовые офисные расчёты и разбор ошибок новичка · Python 3

Публичная версия **Обновлено 09.11.2026** **Артур Хант — ассистент эксперта**

Содержание

Зачем Python офисному работнику **2**

Как установить Python **3**

Как запустить код из шпаргалки **4**

Часть 1. Переменные, типы, операции и константы **5**

Часть 2. Функции, аргументы и возвращаемые значения **8**

Python в офисе: готовые примеры **12**

Частые ошибки новичка и что с ними делать **13**

Отдельно для каждой группы читателей **13**

Где учиться дальше бесплатно **21**

Об этом документе

Как пользоваться шпаргалкой. Каждый урок устроен одинаково: сначала правило, затем пример кода и результат. Прочитайте правило, закройте ответ в примере и попробуйте предсказать, что выведет программа, — а потом сверьтесь. Код набран моноширинным шрифтом на тёмном фоне; после знака # — комментарий с результатом. Уроки идут по порядку: присваивание → типы → операции → функции → аргументы → результат. Шпаргалка написана для семи групп читателей: офисных работников — экономистов, юристов, руководителей и инженеров, в том числе в крупных промышленных и нефтегазовых компаниях; IT-специалистов; фрилансеров и удалённых специалистов; школьников 9–11 классов; студентов; учителей и преподавателей вузов; врачей и медсестёр. У каждой группы есть свой раздел.

Зачем **Python** офисному работнику

1. **Excel и выгрузки.** Python за секунды складывает сотни строк из выгрузок 1С и Excel, объединяет десятки файлов в один, ищет расхождения между таблицами.
2. **Рутина с файлами.** Переименовать сотни сканов по шаблону, разложить документы по папкам, найти файлы по дате — то, что вручную занимает часы.
3. **Отчёты и расчёты.** НДС, премии, суточные, рабочие дни до срока, графики — один раз написали программу, дальше пользуетесь годами.
4. **Карьера.** Знание Python выделяет бухгалтера, экономиста, аналитика, кадровика или менеджера в вакансиях — в Москве, Петербурге, городах-миллионниках и в городах поменьше, где таких специалистов особенно мало.
5. **Письма и документы по шаблону.** Письма-напоминания контрагентам по списку из Excel, типовые ответы, реестр договоров по файлам в папке — текст заполняется сам, остаётся проверить и отправить.
6. **Проверка данных до отчёта.** Найти пустые ячейки, дубли номеров договоров или ИНН, суммы, записанные текстом, — до того, как отчёт уйдёт руководителю или в налоговую.

7. **Бесплатно и на своём компьютере.** Python — бесплатная программа с открытым кодом. Программы работают локально: файлы с коммерческой тайной и персональными данными никуда не отправляются.

8. **Не нужно становиться программистом.** Для офисных задач хватает основ из этой шпаргалки: переменные, функции, циклы и несколько модулей. Готовые примеры в конце документа занимают 10–20 строк.

Задача из офиса	Чем решается в Python	Нужно ли что-то ставить
Сложить суммы из выгрузки CSV, найти итоги по отделам	Модуль csv	Нет — входит в Python
Найти, переименовать, разложить файлы по папкам	Модули pathlib и shutil	Нет — входит в Python
Посчитать сроки, рабочие дни, возраст задолженности	Модуль datetime	Нет — входит в Python
Округлить деньги точно до копейки	Модуль decimal	Нет — входит в Python
Собрать данные из десятков файлов в один	Модули pathlib и csv	Нет — входит в Python
Читать и заполнять файлы Excel .xlsx	Библиотека openpyxl	Да: python -m pip install openpyxl
Сводные таблицы и сверка больших реестров	Библиотека pandas	Да: python -m pip install pandas
Графики для отчёта и презентации	Библиотека matplotlib	Да: python -m pip install matplotlib

Как установить **Python**

1. **Скачайте Python.** Откройте официальный сайт python.org, раздел Downloads, и скачайте последнюю версию Python 3 для Windows или macOS. Сайт доступен из России, установка бесплатна.

2. **Windows: поставьте галочку.** В первом окне установщика отметьте «Add python.exe to PATH» и нажмите «Install Now». Без этой галочки Python не будет запускаться из командной строки.

3. **macOS.** Запустите скачанный установщик и пройдите все шаги. После установки в папке «Программы» появится папка Python 3 с редактором IDLE.

4. **Рабочий компьютер.** Если у вас нет прав администратора, попросите ИТ-отдел установить Python. Это обычная бесплатная программа.

5. **Удобный редактор для новичка.** Вместе с Python ставится простой редактор IDLE — его достаточно. Можно поставить и бесплатный редактор Thonny (thonny.org), сделанный специально для начинающих.

6. **Windows: Python install manager.** С версии 3.14 документация Python рекомендует ставить Python на Windows через Python install manager — он есть на python.org/downloads и в Microsoft Store. Классический установщик пока выкладывается, но объявлен устаревшим: для Python 3.16 и новее его выпускать не будут. Команда py list покажет, какие версии установлены.

7. **Проверьте установку.** Откройте командную строку (Windows: «Пуск» → наберите cmd) или Терминал (macOS) и введите py -V или python -V на Windows, python3 -V на macOS. Ответ вида «Python 3.15.2» значит, что всё готово.

8. **Библиотеки для Excel.** Модули csv, pathlib и datetime из уроков уже входят в Python. Для файлов .xlsx поставьте библиотеку openpyxl той же командной строкой: python -m pip install openpyxl на Windows, python3 -m pip install openpyxl на macOS.

Что видите	Почему так	Что делать
«python» не является внутренней или внешней командой	Python не добавлен в PATH при установке	Запускайте через py или переустановите с галочкой «Add python.exe to PATH»

Что видите	Почему так	Что делать
Вместо Python открывается Microsoft Store	Сработал псевдоним Windows, а сам Python не установлен	Установите Python с python.org или в «Параметрах» Windows отключите «Псевдонимы выполнения приложений» для python
pip: «не является командой»	pip вызывают напрямую, а не через Python	Пишите python -m pip install … (на macOS — python3 -m pip install …)

Как запустить код из шпаргалки

1. **Откройте IDLE** (в меню «Пуск» на Windows или в папке Python 3 на Mac).
2. **Создайте файл:** меню File → New File. Откроется пустое окно для программы.
3. **Наберите код** из урока. Набирайте руками, а не копируйте: так быстрее запоминается.
4. **Сохраните файл** (File → Save) с именем латиницей и окончанием .py, например urok1.py.
5. **Запустите:** Run → Run Module или клавиша F5. Результат появится в окне Shell.
6. **Окно Shell — для проб.** После значка >>> можно набрать одну строку, например 120000 * 0.22, и сразу увидеть ответ. То, что набрано в Shell, не сохраняется, поэтому программу пишите в файле.
7. **Файлы — рядом с программой.** Если программа читает otchet.csv, положите этот файл в папку, где сохранён .py, или укажите полный путь. Узнать, в какой папке программа ищет файлы, помогут две строки: import os и print(os.getcwd()).
8. **Запуск без IDLE.** Откройте командную строку в папке с программой и наберите py urok1.py (Windows) или python3 urok1.py (macOS). Так удобно запускать готовую программу каждый месяц на новой выгрузке.
9. **Программа зависла.** Нажмите Ctrl+C в окне Shell — выполнение прервётся. Частая причина — цикл while, условие которого никогда не становится ложным.

Клавиши и меню IDLE	Что делают
F5 (Run → Run Module)	Сохраняет изменения и запускает программу из файла
Alt+P и Alt+N (на macOS — Ctrl+P и Ctrl+N)	Предыдущая и следующая команда в окне Shell — не нужно набирать заново
Tab или Ctrl+Пробел	Подсказка: варианты окончания имени переменной, функции или модуля
Ctrl+C в окне Shell	Прервать выполнение программы
Ctrl+Z (на macOS — Cmd+Z)	Отменить последнее изменение в тексте программы
Options → Configure IDLE → Keys	Посмотреть все сочетания клавиш вашей версии IDLE
Format → Comment Out Region	Закомментировать выделенные строки, чтобы временно их отключить; Uncomment Region — вернуть

Часть 1. Переменные, типы, операции и константы

01 Как читать и выполнять код

Python выполняет инструкции по порядку. В учебных программах пишите одно действие на строке. Выражение вычисляет значение; инструкция выполняет действие. Вывод появляется после вызова print().

```
product = 'Малина'  # присвоить строку
quantity = 15       # присвоить число
print(product)     # Малина
print(quantity)    # 15
```

Литерал — значение прямо в коде: 15, 3.14, 'Малина'. **Имя** — обозначение связанного с ним значения: quantity. **Выражение** — например quantity + 10.

Комментарий: # вне строки начинает примечание до конца строки. Закомментировать код — добавить #; раскомментировать — убрать его. В '#' решётка является текстом.

Порядок изучения: присваивание → типы → операции → функции → аргументы → результат. Прочитайте правило, закройте ответ примера и предскажите вывод.

02 Переменные и присваивание

имя = выражение: сначала Python вычисляет правую часть, затем связывает имя слева с результатом. Знак = означает присваивание, а не проверку равенства.

```
count = 15
total = count + 10  # total: 25
count = 20         # total всё ещё 25
count = count + 1  # count: 21
```

Новое присваивание заменяет связь имени. Значение total не пересчитывается: это готовый результат.

`count = 15` → `15` `count = 20` → `20`

Имена: осмысленные английские слова, snake_case: jar_count, jar_volume. Регистр важен: count и Count — разные имена. Не начинайте имя с цифры; не используйте пробелы и дефисы.

Избегайте: непонятных a, nm, транслита banka и имён встроенных функций print, int, str, type. До чтения имени ему должно быть присвоено значение.

03 Типы данных и типизация

str — строка: 'Тоня', "текст", '31', ''. Кавычки по краям должны совпадать. **int — целое число:** 15, 0, -2. **float — число с плавающей точкой:** 3.14, 0.001; разделитель — точка.

```
print(type('31'))  # <class 'str'>
print(type(31))    # <class 'int'>
print(type(3.14))  # <class 'float'>
```

Динамическая типизация: тип принадлежит объекту; одно имя можно последовательно связать со значениями разных типов: x = 5, затем x = 'текст'.

Сильная типизация: Python не складывает несовместимые типы по догадке. Выражение 1 + '2' вызывает TypeError. Выберите: складывать числа или объединять строки.

Уточнение: совместимые числовые типы могут участвовать в общей операции: 1 + 2.5 даёт 3.5. Поэтому правило «Python никогда сам не преобразует типы» слишком категорично. Для проверки неожиданного значения временно выведите **print(x, type(x))**. Число 31 и строка '31' печатаются похоже, но ведут себя по-разному.

04 Операции с числами и строками

```
2 + 3          # 5, сложение
7 – 2          # 5, вычитание
3 * 4          # 12, умножение
7 / 2          # 3.5, деление
3 ** 2         # 9, степень
2 + 3 * 4      # 14
(2 + 3) * 4    # 20
```

В этих формулах: скобки задают группировку; степень выполняется раньше умножения и деления, затем идут сложение и вычитание. / даёт float даже при делении нацело: 4 / 2 → 2.0.

```
'перс' + 'тень' # 'перстень'
'ля' * 3       # 'ляляля'
'25' + '50'    # '2550', не 75
'А' + ' ' + 'Б' # 'А Б'
```

+ для строк — конкатенация (склеивание), *** с целым числом** — повторение. Пробелы при склеивании сами не добавляются. Вычитание строк и сложение строки с числом вызывают TypeError.

05 Преобразование типов

str(x) создаёт строковое представление; **int(x)** преобразует подходящее значение в целое число. Результат нужно сохранить или сразу использовать.

```
number = 100
text = str(number)  # '100'
print(type(number)) # <class 'int'>
print(type(text))   # <class 'str'>
print(int('25') + int('50')) # 75
```

Вызов str(number) сам по себе не меняет number. Если написать number = str(number), имя будет связано уже со строкой.

```
int('123')     # 123
int('123abc')  # ValueError
int('3.14')    # ValueError
float('3.14')  # 3.14
int(3.9)       # 3, не округление
```

Выбор преобразования: для расчётов переводите числовой текст в число; для склеивания с текстом — число в строку. float() и поведение int() с дробями здесь даны как уточнение к материалам курса.

06 Вывод через print

print(a, b, c) печатает значения разных типов, по умолчанию разделяя их пробелом и завершая вывод переводом строки. Преобразование в str() для отдельных аргументов не нужно.

```
count = 15
print('Банок:', count)
# Банок: 15
print('Банок: ' + str(count))
# Банок: 15
```

Запятая и + решают разные задачи. Запятая разделяет аргументы print(); + сначала создаёт одну строку. print('Банок: ' + count) неверно: ошибка возникает ещё до печати.

```
berries = 25
vegetables = '12'
total = berries + int(vegetables)
print('Всего заготовлено', total,
      'банок.')
# Всего заготовлено 37 банок.
```

Внутри скобок разрешены переносы строк. В тренажёре важны точные пробелы, регистр и знаки препинания.

07 Константы и формула объёма

Константа — имя для значения, которое по смыслу не должно меняться. Имена: UPPER_SNAKE_CASE. Python не запрещает менять константы.

```
PI = 3.14
CM3_TO_LITERS = 0.001
height = 9
radius = 5
volume = (PI * height * radius ** 2
          * CM3_TO_LITERS)
```

`V = π × h × r²`

`см³ × 0.001 = литры`

Результат: 0.7065 л. Если h и r в сантиметрах, π × h × r² даёт см³. Один литр — 1000 см³, поэтому умножаем на 0.001. Нужен радиус, не диаметр.

Польза: смысл чисел понятен; правка в одном месте. 3.14 — приближение π.

Python 3 • Читайте блоки слева направо, затем сверху вниз • Печать A4, масштаб 100%

08 Проверка первого листа

Предскажите результат без запуска. ① '7' + '3' ② int('7') + 3 ③ 'ха' * 2 ④ type('3.14') ⑤ 2 + 3 ** 2. Затем проверьте себя ниже.

Найдите ошибку: ⑥ print('Всего: ' + 12) ⑦ int('два') ⑧ print(unknown), если unknown ранее не определено.

Ответы: ① '73' ② 10 ③ 'хаха' ④ <class 'str'> ⑤ 11. ⑥ TypeError: используйте str(12) или отдельный аргумент. ⑦ ValueError: строка не задаёт целое число. ⑧ NameError: имя неизвестно.

Чек-лист: строка в кавычках; число без кавычек; дробь через точку; имя задано до использования; типы совместимы; при + между словами есть пробелы.

Разбор ошибки: прочитайте последнюю строку сообщения, найдите указанную строку кода, проверьте значения и типы, исправьте причину и запустите заново.

Важно для новичка: комментарии с ответами не выполняются. Не переписывайте номера строк и кнопки «Запустить код» из тренажёра в программу.

Часть 2. Функции, аргументы и возвращаемые значения

09 Функция и её вызов

Функция — именованный блок кода для одной задачи. Принцип **DRY** (Don’t Repeat Yourself) предлагает вынести повторяющиеся действия в функцию и вызывать её с разными данными.

```
def welcome():
    print('Добро пожаловать!')
welcome()  # выполнить тело
welcome()  # выполнить ещё раз
```

Синтаксис: def, пробел, имя, круглые скобки, двоеточие. Тело пишется с отступом в 4 пробела. Возврат к прежнему отступу завершает тело. Не смешивайте табуляцию и пробелы.

Объявление не выполняет тело. Вызов — имя со скобками: welcome(). Просто welcome обозначает саму функцию. К моменту вызова определение функции должно быть выполнено.

Имена действий: calculate_volume, show_message, get_total. Используйте snake_case; не занимайте имена print и других встроенных функций.

10 Параметры и аргументы

Параметр — имя в объявлении функции. **Аргумент** — конкретное значение, переданное при вызове. Параметры получают значения аргументов и используются в теле.

```
def welcome(profession, name):
    print('Привет,', profession, name)
welcome('фермер', 'Тоня')
# Привет, фермер Тоня
```

`'фермер'` → `profession` `'Тоня'` → `name`

Аргументом может быть литерал, значение переменной, результат выражения или вызова другой функции. Имя переменной при вызове не обязано совпадать с именем параметра.

```
user_name = 'Женя'
welcome('садовод', user_name)
```

Если обязательный аргумент пропущен, Python выдаст TypeError. Лишний аргумент тоже может вызвать TypeError. Проверьте имена, порядок и число параметров.

11 Как передавать аргументы

```
def action(name, food='ягоды'):
    print(name, 'заготавливает', food)
action('Женя', 'морковь')
action('Женя', food='морковь')
action(food='морковь', name='Женя')
```

Все три вызова печатают одно и то же. **Позиционные** аргументы сопоставляются слева направо. **Именованные** задают параметр явно через имя=значение; их порядок можно менять.

```
action('Женя')  # food: 'ягоды'
action(name='Женя')  # то же самое
action('картофель')  # это имя!
```

В обычных вызовах этого курса: сначала позиционные аргументы, затем именованные. action(name='Женя', 'морковь') — SyntaxError. action('Женя', name='Тоня') — TypeError: name задан дважды. Именованный аргумент должен совпадать с именем параметра. Значение по умолчанию для этого не обязательно. У некоторых встроенных функций есть ограничения на именованные аргументы.

12 Умолчания и локальные имена

Значение по умолчанию задаётся в объявлении: def action(name, food='ягоды'). Параметр name обязателен; food можно пропустить. Переданный аргумент заменяет значение по умолчанию. В показанной форме объявления обязательные параметры идут перед необязательными. def

action(food='ягоды', name): — SyntaxError. При вызове с одним позиционным аргументом он попадёт в name.

```
def make_message(name):
    message = 'Привет, ' + name
    return message
result = make_message('Тоня')
print(result)  # Привет, Тоня
```

Локальные имена: name и message принадлежат вызову функции. print(message) снаружи вызовет NameError, если снаружи такого имени нет.

Как получить данные наружу: вернуть значение через return и сохранить результат вызова. Локальным является имя параметра, а не обязательно само переданное значение.

13 Возвращаемое значение

```
def plus_five(value):
    return value + 5
result = plus_five(2)  # result: 7
print(result)         # 7
print(plus_five(10))  # 15
```

return выражение вычисляет выражение, завершает текущий вызов и передаёт значение в место вызова. Можно вернуть литерал, значение переменной или результат другого вызова.

print показывает данные человеку; **return** отдаёт данные программе. Функция может возвращать результат без печати. Присваивание result = plus_five(2) само ничего не выводит.

```
def show(value):
    print(value)
answer = show(7)  # печатает 7
print(answer)    # None
```

Уточнение: без return или при пустом return функция возвращает None — отсутствие содержательного результата. Код после выполненного return в этом вызове уже не исполняется.

14 Вложенные вызовы и заглушки

```
def plus_five(value):
    return value + 5
def multiply_by_three(value):
    return value * 3
result = plus_five(multiply_by_three(7))
print(result)  # 26
```

`7` → `× 3 → 21` → `+ 5` → `26`

Сначала multiply_by_three(7) возвращает 21; затем plus_five(21) возвращает 26. Значение сохраняется в result и передаётся в print.

```
def later():
    pass  # ничего не делать
def another():
    ...   # значение Ellipsis
```

Пустое тело недопустимо. pass — инструкция-заглушка; ... — отдельное значение Ellipsis, которое тоже подходит как заглушка. Обе эти функции при вызове возвращают None.

15 Программа расчёта объёма

```
PI = 3.14
CM3_TO_LITERS = 0.001
def calculate_volume(height, radius):
    return (PI * height * radius ** 2
            * CM3_TO_LITERS)
first = calculate_volume(9, 5)
second = calculate_volume(15, 7)
third = calculate_volume(20, 9)
print('Первая:', round(first, 2), 'л')
print('Вторая:', round(second, 2), 'л')
print('Третья:', round(third, 2), 'л')
```

Вывод: Первая: 0.71 л; Вторая: 2.31 л; Третья: 5.09 л — каждое сообщение на отдельной строке.

Разбор: константы задают общие коэффициенты; параметры получают размеры; return отдаёт объём; print выводит подпись и результат. Формула написана один раз.

Уточнение к уроку: round(x, 2) округляет до двух знаков после точки; в исходной формуле округление заявлено, но не выполнено. round() не гарантирует два видимых знака: 2.0 не станет 2.00.

Python 3 • Читайте блоки слева направо, затем сверху вниз • Печать A4, масштаб 100%

16 Ошибки и самопроверка

SyntaxError: проверьте кавычки, скобки, двоеточие после def и порядок аргументов. **IndentationError:** проверьте отступ тела.

NameError: имя не определено или недоступно здесь.

TypeError: несовместимые типы, недостающий/лишний аргумент, двойное значение параметра. **ValueError:** тип аргумента подходит, но значение нельзя обработать, как в int('кот').

Проверьте себя: ① Чем параметр отличается от аргумента? ② Что напечатает print(plus_five(3))? ③ Как передать только food при def action(name='Гость', food='ягоды')? ④ Что вернёт функция с pass?

Ответы: ① Имя в def и значение при вызове. ② 8. ③ action(food='яблоки'). ④ None. Путь данных: аргументы, параметры, вычисление, return, вывод.

Основа: семь приложенных уроков — «Добро пожаловать», «Переменные и их значения», «Типизация и типы данных», «Константы», «Функции», «Параметры и аргументы функций», «Возвращаемое значение».

Дальше по программе курса: ветвления, циклы, коллекции; затем окружение и Git, далее ООП. Эти темы во вложениях анонсированы, но ещё не разобраны.

Python в офисе: готовые примеры

Примеры проверены запуском в Python 3. Работайте с копиями файлов, пока не убедитесь, что программа делает то, что нужно.

П1 НДС: выделить из суммы и начислить сверху

Ставку храним в переменной — если она изменится, правим одну строку. С 2026 года основная ставка НДС в России — 22%. Запись **:.2f** внутри f-строки выводит число с двумя знаками после запятой.

```
stavka = 22                    # ставка НДС в процентах
summa_s_nds = 122000
nds = summa_s_nds * stavka / (100 + stavka)
print(f"НДС в сумме: {nds:.2f} руб.")          # НДС в сумме: 22000.00 руб.
bez_nds = 50000
s_nds = bez_nds * (1 + stavka / 100)
print(f"С НДС: {s_nds:.2f} руб.")   # С НДС: 61000.00 руб.
```

П2 Суточные за командировку – функция с умолчанием

Функция принимает число дней и норму в рублях. Если норму не указать, берётся значение по умолчанию.

```
def sutochnye(dni, norma=700):
    return dni * norma

print(sutochnye(5))            # 3500
print(sutochnye(5, norma=1000)) # 5000
```

П3 Премия от оклада

round() округляет до целых рублей. Подставьте свой оклад и процент.

```
def premiya(oklad, procent):
    return round(oklad * procent / 100)

print(premiya(85000, 15))      # 12750
```

Частые ошибки новичка и что с ними делать

Сообщение Python	Что случилось	Что делать
IndentationError	Неправильный отступ: строки внутри функции или условия должны начинаться с 4 пробелов	Выровняйте отступы; не смешивайте пробелы и табуляцию
SyntaxError	Забыта кавычка, скобка или двоеточие в конце строки с def, if, for	Посмотрите на строку, которую показал Python, и на строку перед ней
NameError	Опечатка в имени переменной или функции, либо её ещё не создали	Сравните написание; имена различают большие и маленькие буквы
TypeError	Смешали разные типы: например, «5» + 5 — текст и число	Переведите текст в число через int() или float()
ValueError: could not convert string to float: '12,5'	Русская запятая в дробном числе — Python понимает только точку	Замените запятую: float('12,5'.replace(',', '.'))
UnicodeDecodeError	Файл из Excel или 1С сохранён в кодировке Windows-1251	Откройте файл с encoding='cp1251' вместо 'utf-8'
FileNotFoundError	Python не нашёл файл: опечатка в имени или файл лежит в другой папке	Проверьте имя и расширение; положите файл рядом с программой или укажите полный путь
PermissionError	Файл открыт в Excel или нет прав на запись в папку	Закройте файл в Excel и запустите программу снова
KeyError: 'Сумма'	В CSV нет столбца с таким заголовком: лишний пробел, другая буква или невидимый символ в начале файла	Выведите print(stroka.keys()) и сравните; невидимый символ уберёт encoding='utf-8-sig'
ModuleNotFoundError: No module named 'openpyxl'	Библиотека не установлена	python -m pip install openpyxl (на macOS — python3 -m pip ...)
ZeroDivisionError	Деление на ноль: например, план отдела равен 0	Перед делением проверьте делитель: if plan != 0
IndexError: list index out of range	Обращение к элементу, которого нет: в строке меньше столбцов, чем ожидалось	Проверьте len(stroka); пропускайте пустые строки

Как читать сообщение об ошибке: смотрите на последнюю строку — там тип ошибки и пояснение. Чуть выше указан файл и номер строки (line 12), где Python остановился. Саму ошибку часто нужно искать в этой строке или строкой выше.

Отдельно для каждой группы читателей

Дальше — по странице на каждую группу читателей: что важно именно вам и с чего начать. Найдите свою карточку; остальные разделы документа общие для всех.

В этом разделе — по странице на каждый пункт:

1. Экономистам, юристам, руководителям и инженерам
2. IT-специалистам
3. Фрилансерам и удалённым специалистам

4. Школьникам 9–11 классов и студентам
5. Учителям и преподавателям вузов
6. Врачам и медсёстрам

Экономистам, юристам, руководителям и инженерам

Экономисты, финансисты, юристы, руководители и топ-менеджеры, инженеры крупных промышленных и нефтегазовых компаний

22%
ставка НДС с 2026 года – в примере П1

1 строка
менять, если изменится ставка

5 минут
на программу, которая заменит часы вручную

Что важно знать

Экономистам и финансистам

Сводные таблицы по выгрузкам 1С, сверка двух реестров, расчёт отклонений план – факт, отчёты, которые раньше собирались вручную по дням. Дальше стоит освоить библиотеку pandas – «Excel в Python».

Юристам

Переименовать и разложить по папкам сотни документов, составить реестр договоров по файлам, найти документы по ключевым словам – на своём компьютере, без отправки в облако.

Руководителям

Не обязательно программировать самому: понимание основ помогает ставить аналитикам реальные задачи, оценивать сроки и видеть, какую рутину в отделе можно автоматизировать.

Инженерам промышленных и нефтегазовых компаний

Расчёты по формулам, обработка выгрузок с датчиков и измерений в CSV, пересчёт единиц, графики для отчётов. Для работы с данными объектов пользуйтесь рабочим компьютером и правилами вашей службы безопасности.

Делайте

- Пройдите уроки 1–8 по порядку
- Повторите примеры на своих цифрах
- Описывайте рутину шагами – это будущая программа

Не делайте

- Не копируйте код – набирайте руками
- Не запускайте программы на оригиналах файлов
- Не храните реальные данные в учебных файлах

Инструкция: с чего начать

1. Установите Python и пройдите уроки 1–8 – это основа любого расчёта.
2. Повторите пример с НДС и премией на своих цифрах.
3. Выберите одну еженедельную рутину и опишите её шагами – это будущая программа.

Где читать подробнее в этом документе

Python в офисе: готовые примеры | **Как установить Python**

Частые ошибки новичка и что с ними делать

IT-специалистам

Разработчики, системные администраторы, тестировщики, аналитики, специалисты техподдержки

16 уроков	PEP 8	0 зависимостей
базовый конспект для онбординга	стиль, к которому приучают уроки	только стандартный Python

Что важно знать

Для онбординга коллег

Шпаргалка подойдёт как опорный конспект для новичков в команде, тестировщиков и аналитиков, которым нужен базовый Python.

Администраторам и тестировщикам

Скрипты для автоматизации проверок, обработки журналов и выгрузок — следующий шаг после уроков с модулями.

Стиль кода

Уроки о читаемости, отступах и именах соответствуют общепринятому стилю PEP 8 — его ждут на код-ревью.

Делайте

- Давайте шпаргалку новичкам и смежным отделам
- Начинайте с примеров из «Python в офисе»
- Проводите вместе разбор ошибок

Не делайте

- Не перегружайте новичков библиотеками сразу
- Не давайте рабочие данные для учебных задач
- Не пропускайте разбор ошибок

Инструкция: с чего начать

① Пролистайте уроки и отметьте, что нужно новичкам в вашей команде.

② Используйте шпаргалку как опорный конспект для онбординга.

③ Предложите коллегам из других отделов автоматизировать одну рутину вместе.

Где читать подробнее в этом документе

Частые ошибки новичка и что с ними делать

Как запустить код из шпаргалки | **Где учиться дальше бесплатно**

Фрилансерам и удалённым специалистам

Фрилансеры, digital-специалисты, удалённые сотрудники

Kwork, FL.ru, Хабр Фриланс
биржи, где заказывают скрипты

1 скрипт
в портфолио лучше строчки «знаю Python»

Вымышленные данные
для отладки

Что важно знать

Автоматизация своей работы

Скрипты экономят часы на рутине: переименование и сортировка файлов клиентов, сбор отчётов, подготовка выгрузок.

Новая услуга

Небольшие скрипты для малого бизнеса — обработка таблиц, выгрузок маркетплейсов, отчётов — частый заказ на российских биржах фриланса (Kwork, FL.ru, Хабр Фриланс).

Данные клиентов

Отлаживайте скрипты на вымышленных данных и не загружайте файлы заказчика в онлайн-сервисы без его согласия.

Делайте

- Автоматизируйте свою рутину первой
- Оформите готовый скрипт как услугу
- Пишите заказчику инструкцию по запуску

Не делайте

- Не берите заказы на обработку данных без договора
- Не загружайте файлы клиентов в онлайн-сервисы
- Не обещайте сложное после первых уроков

Инструкция: с чего начать

① Пройдите уроки по порядку, набирая код руками.

② Автоматизируйте одну свою рутинную задачу.

③ Добавьте получившийся скрипт в портфолио как пример услуги.

Где читать подробнее в этом документе

Python в офисе: готовые примеры | **Частые ошибки новичка и что с ними делать**

Где учиться дальше бесплатно

Школьникам 9–11 классов и студентам

Школьники 9–11 классов и студенты вузов

27 заданий
ЕГЭ по информатике, 3 ч 55 мин

Python
основной язык для решения задач ЕГЭ

ОГЭ
тоже с заданиями на программирование

Что важно знать

ЕГЭ по информатике

Экзамен сдаётся на компьютере: 27 заданий за 3 часа 55 минут, максимум 29 первичных баллов. Основной язык для решения задач – Python: многие задания быстрее и надёжнее решать короткой программой, чем вручную. Уроки этой шпаргалки – фундамент для таких программ.

ОГЭ в 9 классе

В ОГЭ по информатике тоже есть задания на программирование – Python подходит и для них.

Как готовиться

Решайте задания из открытого банка и демоверсий ФИПИ, проходите курсы по ЕГЭ на Stepik и проверяйте себя по таймеру. Набирайте код руками, а не копируйте.

Олимпиады

Всероссийская олимпиада школьников по информатике и перечневые олимпиады дают льготы при поступлении – вплоть до зачисления без экзаменов.

Студентам

Python пригодится далеко не только программистам: обработка данных в курсовых и лабораторных, графики для диплома, автоматизация рутины. Экономистам, социологам, психологам и биологам он часто нужен уже на 2–3 курсе.

Стажировки

Даже простой проект на Python – например, программа из раздела «Python в офисе», доработанная под ваши задачи, – выглядит в резюме сильнее, чем строчка «знаю Python».

Делайте

- Решайте задания ФИПИ программой
- Тренируйтесь по таймеру
- Студентам – применяйте Python в лабораторных

Не делайте

- Не заучивайте код без понимания
- Не готовьтесь только по ответам ИИ
- Не откладывайте практику до 11 класса

Инструкция: с чего начать

1. Установите Python и пройдите уроки по порядку.
2. Решите 5 заданий ЕГЭ или ОГЭ по информатике программой.
3. Студентам – примените Python в ближайшей лабораторной или курсовой.

Где читать подробнее в этом документе

Урок 1: как читать и выполнять код | **Частые ошибки новичка и что с ними делать** | **Где учиться дальше бесплатно**

Учителям и преподавателям вузов

Учителя школ, преподаватели колледжей и вузов

random
варианты задач с разными числами

CSV
выгрузка журнала – для своих расчётов

Порядок уроков
как в школьной программе

Что важно знать

Ведомости и журналы

Выгрузку оценок из электронного журнала можно сохранить как CSV и посчитать средние, найти пропуски и отстающих программой из раздела «Python в офисе» – на своём компьютере, не отправляя данные учеников в облако.

Варианты заданий

Модуль random помогает генерировать индивидуальные варианты задач с разными числами – так меньше списывания.

Учителям информатики

Уроки шпаргалки идут в том же порядке, что и школьная программа: присваивание, типы, операции, функции. Их удобно давать ученикам как опорный конспект перед ОГЭ и ЕГЭ.

Преподавателям вузов

Python – стандарт для обработки данных исследований: статистика, графики для статей, автоматизация рутинных расчётов.

Делайте

- Давайте шпаргалку как опорный конспект
- Генерируйте индивидуальные варианты
- Считайте оценки на своём компьютере

Не делайте

- Не отправляйте выгрузки журнала в облако
- Не давайте задания без разбора ошибок
- Не пропускайте уроки о типах данных

Инструкция: с чего начать

① Дайте ученикам шпаргалку как опорный конспект.

② Придумайте задание с генерацией вариантов через random.

③ Обработайте выгрузку оценок на своём компьютере.

Где читать подробнее в этом документе

Python в офисе: готовые примеры | Частые ошибки новичка и что с ними делать | Как запустить код из шпаргалки

Врачам и медсёстрам

Врачи, медсёстры и другие медицинские работники

Обезличенные данные
только они – для научной работы

Графики дежурств
считаются за минуты

Медицинская информатика
растущее направление

Что важно знать

Таблицы и графики дежурств

Python поможет свести расписание, посчитать часы и смены – на рабочем компьютере и по правилам вашей организации.

Научная работа

Для диссертации и статей Python обрабатывает выборки, строит графики и считает статистику. Работайте только с обезличенными данными и в одобренной среде.

Врачебная тайна

Не загружайте таблицы с данными пациентов в онлайн-сервисы и облачные ИИ, даже чтобы «просто проверить код». Отлаживайте программу на вымышленных данных.

Медицинская информатика

Знание Python открывает путь в аналитику здравоохранения и цифровую медицину – направление, где специалистов с медицинским образованием особенно не хватает.

Делайте

- Отлаживайте программы на вымышленных данных
- Пользуйтесь рабочим компьютером и правилами организации
- Для статистики продолжите обучение

Не делайте

- Не загружайте таблицы с данными пациентов в онлайн-сервисы
- Не публикуйте выборки с персональными данными
- Не обходите правила службы безопасности

Инструкция: с чего начать

1. Установите Python и пройдите уроки 1–8.
2. Посчитайте часы и смены по своему графику на вымышленных данных.
3. Для научной работы – продолжите обучение с библиотеками для статистики.

Где читать подробнее в этом документе

Как установить Python | Python в офисе: готовые примеры | Где учиться дальше бесплатно

Где учиться дальше бесплатно

1. **«Поколение Python: курс для начинающих» на Stepik** — бесплатный русскоязычный курс с сотнями задач и автоматической проверкой.
2. **Хендбук «Основы Python» от Яндекс Образования** — бесплатный учебник с задачами на сайте education.yandex.ru.
3. **Официальная документация** на docs.python.org — справочник по языку и стандартной библиотеке.
4. **Режим занятий.** 20–30 минут в день лучше, чем 4 часа раз в неделю. Сразу пробуйте применять уроки к своим рабочим таблицам и файлам — на копиях.
5. **«Программирование на Python» на Stepik** (stepik.org/course/67) — ещё один бесплатный вводный курс на русском: переменные, списки, условия, циклы и много задач с проверкой.
6. **Официальный учебник Python** (раздел Tutorial на docs.python.org) — на английском; для чтения помогает встроенный в браузер перевод страницы.
7. **Как искать ответ на ошибку.** Скопируйте в поиск последнюю строку сообщения Python, например «KeyError python csv», — почти всегда кто-то уже разобрал такую же ситуацию.
8. **Своя мини-задача.** Выберите одну рутинную операцию из своей недели — например, сводку по выгрузке — и доведите программу до рабочего состояния. Одна законченная задача учит больше, чем десяток прочитанных глав.
9. **Повторение.** Через неделю решите пройденные уроки заново по памяти, не заглядывая в шпаргалку, — так сразу видно, что усвоено, а что стоит перечитать.

Неделя	Что изучить	Задача из своей работы
1	Переменные, типы, f-строки, арифметика	Посчитать НДС, премию или суточные для своих чисел
2	Функции, аргументы, return	Оформить каждый расчёт как функцию и вызвать её для нескольких сотрудников
3	Списки, словари, циклы, условия	Найти итоги по отделам и строки с пустой суммой в небольшом списке
4	Модули csv, pathlib, datetime	Обработать копию своей выгрузки из Excel или 1С и сохранить итог в CSV
5–6	Библиотека openpyxl или pandas	Читать файлы .xlsx сразу, без пересохранения в CSV

СВЯЗЬ И НОВЫЕ МАТЕРИАЛЫ

Где искать новые материалы

Новые файлы, а также ссылки на сообщества во ВКонтакте, Telegram, Teletype и Boosty — и, возможно, на YouTube, VK Видео, Rutube, Stepik и Udemy — ищите по этим контактам.

TG **TELEGRAM** t.me/artur_khant	VK **ВКОНТАКТЕ** vk.ru/artur_khant
TT **TELETYPE** teletype.in/@artur_khant	IG **INSTAGRAM** instagram.com/artur_khant
GH **GITHUB** github.com/artur-khant	CV **САЙТ-РЕЗЮМЕ** arturishkildin.cv

Не нашли ссылку на хранилище файлов или нужный цифровой продукт?
Напишите мне в личные сообщения или просто наберите в поиске

АРТУР ХАНТ